



FOR SPRING DEVELOPERS MOVING TO .NET

# The DI Validation Trap

## Spring Boot to .NET, the flagship

It failed in Development and started in Production. Nothing changed but the environment.

One payment service written twice, in Spring Boot and in ASP.NET Core on .NET 10 LTS. Controllers, constructor injection and services read almost line for line. Four small mistakes do not behave the same way.

Everything here uses ASP.NET Core's built-in dependency injection container with the normal host defaults.

Every version on this sheet was current on 24 September 2026 and will not stay current. The method is the durable part: run the script, read what came back, and write down the result rather than the expectation.

### In scope

- A service nobody registered, in Development and in Production
- A scoped service inside a singleton
- The two validation flags, and what they do not reject
- Two implementations of one interface
- EF Core: tracking is not saving

### Out of scope

- Performance. Nothing here measures throughput or startup
- Third party containers, which have their own rules
- Anything about how this channel is produced, which is not the subject

Spring's startup failures below are measured on this application. A lazy bean or a programmatic lookup can move when Spring finds a problem too.

## 01 The move, in one table

|                                          |                                                             |
|------------------------------------------|-------------------------------------------------------------|
| A service nobody registered              | Development refuses; Production starts, first request 500   |
| A scoped service in a singleton          | Development refuses; Production keeps the captured instance |
| Two implementations, one dependency      | Starts; single resolves the LAST, the list has both         |
| Change a tracked entity, commit, no save | Not written until SaveChanges                               |

Validation catches the first two rows at startup. It does not fix either, and it does not reject the third, which is valid .NET.



## 02 Failed in Development. Started in Production.

Checkout needs a payment gateway and none is registered. In Development the app refuses to start. Switch the same build to Production and it starts; the first request to checkout returns 500.

The default host enables scope validation and dependency validation only when the environment is Development.

Production, the same build

```
$ ASPNETCORE_ENVIRONMENT=Production dotnet DotnetPayments.dll
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
$ curl -s -o /dev/null -w '%{http_code}' -X POST 'localhost:8194/checkout?amount=2500'
500
```

Development refused to start with "Unable to resolve service for type IPaymentGateway"; Production started and returned 500.

Run on a real machine, 2026-09-24. Reproduce it with `scripts/verify.sh` in the course repository.

"Enables scope validation and dependency validation when the environment is Development."

ASP.NET Core generic host documentation, checked 2026-09-24



### 03 Same ID, every request

AuditLog is registered as a singleton and depends on RequestContext, which is scoped. Built once, the singleton captures the scoped context it was given, so later requests keep seeing that captured instance.

Spring's request scope annotation uses a scoped proxy by default, so a singleton holding it finds the current request on every call. Spring can capture shorter lived beans in other setups; this compares ASP.NET Core scoped with Spring request scope.

```
mistakes/captive: Registrations.cs
```

```
services.AddScoped<RequestContext>();  
services.AddSingleton<AuditLog>();
```

```
Production, two requests
```

```
audit log sees a5e8233b-c621-4bc2-8423-a562a8f3e995,  
  this request is 503119e1-d2f1-46f5-9097-fd3700ac22f1  
audit log sees a5e8233b-c621-4bc2-8423-a562a8f3e995,  
  this request is 012755b8-1fbd-4683-b470-1fe8307cced7
```

The fix is correcting the lifetime relationship. Development refuses this at startup: "Cannot consume scoped service RequestContext from singleton AuditLog".

Production: two requests, one audit id, two request ids. Spring: each request sees its own id.

Run on a real machine, 2026-09-24. Reproduce it with scripts/verify.sh in the course repository.

### 04 Validation fixes detection, not the graph

Switch both checks on everywhere and Production refuses to start for both mistakes, like Development. That moves when you find out. Registering the gateway, and correcting the lifetime relationship, are the fixes.

```
mistakes/validate-always: Program.cs
```

```
builder.Host.UseDefaultServiceProvider(options =>  
{  
    options.ValidateScopes = true;  
    options.ValidateOnBuild = true;  
});
```

With both flags on, Production refused to start for the missing gateway and for the captive context.

Run on a real machine, 2026-09-24. Reproduce it with scripts/verify.sh in the course repository.



## 05 Stripe, then PayPal

Register two gateways and inject one: you get the last one registered. Ask for the list and you get both. It still starts with both validation flags on, because two registrations are valid in the built-in container.

In the Spring app, the equivalent single bean injection refuses to start because two candidates match. You choose with Primary or a Qualifier.

```
● ● ● Production, validation on
```

```
$ curl -s localhost:8216/gateways  
one: PaypalGateway, all: StripeGateway, PaypalGateway
```

"The single IMessageWriter is the last implementation to be registered, whereas the IEnumerable<IMessageWriter> represents all registered implementations."

.NET dependency injection documentation, checked 2026-09-24

Spring: "required a single bean, but 2 were found".

Run on a real machine, 2026-09-24. Reproduce it with scripts/verify.sh in the course repository.



## 06 Tracking is not saving

In Spring, inside a transaction, you can change a loaded entity and never call save: JPA and Hibernate write it at commit. Translate that line for line to EF Core and commit without `SaveChanges`, and the database still has the old amount. EF Core can track and detect the change; it only reaches the database through `SaveChanges`.

```
dotnet-payments: PaymentService.cs

// The Spring method, line for line. Tracked, never saved.
public async Task ChangeAmountLikeSpring(long id, long amount)
{
    await using var tx = await db.Database.BeginTransactionAsync();
    var payment = await db.Payments.FindAsync(id);
    payment!.Amount = amount;
    await tx.CommitAsync();
}

// Tracking is not saving.
public async Task ChangeAmount(long id, long amount)
{
    var payment = await db.Payments.FindAsync(id);
    payment!.Amount = amount;
    await db.SaveChangesAsync();
}
```

Spring `ChangeAmountTest`: written with no save call. .NET `TrackingIsNotSaving`: unchanged until `SaveChanges`.

Run on a real machine, 2026-09-24. Reproduce it with `scripts/verify.sh` in the course repository.

### Before a .NET service goes anywhere near production

1. `ValidateScopes` and `ValidateOnBuild` are true in every environment, set explicitly
2. Every scoped service is consumed only by scoped or transient services
3. Where one interface has several registrations, registration order is deliberate and documented
4. Every write path calls `SaveChanges`, and a test proves the row changed
5. `scripts/verify.sh` passes on a fresh clone



## Everything else

-  **Site**  
<https://code-with-sam-dev.github.io>
-  **YouTube**  
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**  
<https://www.linkedin.com/company/code-with-sam-dev>
-  **X**  
<https://x.com/CodeWithSamDev>
-  **TikTok**  
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**  
<https://www.instagram.com/codewithsamdev>
-  **Facebook**  
<https://www.facebook.com/codewithsamdev>
-  **GitHub**  
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**  
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**  
<https://buymesadza.com/codewithsam>
- The course repository**  
<https://github.com/code-with-sam-dev/spring-to-dotnet>
- ASP.NET Core generic host**  
<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/generic-host>
- .NET dependency injection**  
<https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>
- EF Core: basic save**  
<https://learn.microsoft.com/en-us/ef/core/saving/basic>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. PostgreSQL is a trademark of the PostgreSQL Community Association. .NET and ASP.NET Core are the property of their respective owner. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-24 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.